SO-ARM101

Student Handbook

ROS 2, Gazebo Simulation, and Robot Arm Control

TURTLE Robotics — Software Incubator
Texas A&M University

Supplemental Instruction Series

Table of Contents

Chapter 1 — Introduction and Safety

- 1.1 What You Will Learn
- 1.2 Equipment Overview
- 1.3 Safety Rules

Chapter 2 — Hardware Overview

- 2.1 The SO-101 Arm
- 2.2 Joint Anatomy
- 2.3 Wiring and Power

Chapter 3 — Software Setup

- 3.1 Installing the Environment
- 3.2 Finding Your USB Port
- 3.3 Calibrating the Arm

Chapter 4 — ROS 2 Fundamentals

- 4.1 What is ROS 2?
- 4.2 Nodes
- 4.3 Topics and Messages
- 4.4 Publishers and Subscribers
- 4.5 The ROS 2 Graph
- 4.6 Useful CLI Commands

Chapter 5 — Gazebo Simulation

- 5.1 What is Gazebo?
- 5.2 The TURTLE SI Workspace
- 5.3 Launching the Simulation
- 5.4 Interacting with the World

Chapter 6 — Driving the Turtle Bot

- 6.1 Differential Drive
- 6.2 The cmd_vel Topic
- 6.3 Twist Messages
- 6.4 Sending Commands Manually
- 6.5 Reading Odometry

Chapter 7 — Arm Control via ROS 2 Topics

- 7.1 Joint Position Control
- 7.2 Radians vs Degrees
- 7.3 Joint Topics Reference
- 7.4 Sending Joint Commands
- 7.5 Reading Joint States

Chapter 8 — Week 6 Lab — Open-Loop Drive

- 8.1 Objective

8.2 Background
8.3 Instructions
8.4 Going Further

Chapter 9 — Week 9 Lab — Closed-Loop Control

9.1 Objective
9.2 Background: Odometry
9.3 Instructions
9.4 Going Further

Chapter 10 — Real Hardware — Teleoperation

10.1 Teleop Setup
10.2 Recording Demonstrations

Chapter 11 — Training a Policy

11.1 What is Imitation Learning?
11.2 Running Training
11.3 Evaluating the Policy

Chapter 12 — Troubleshooting and Glossary

12.1 Common Errors
12.2 Glossary

Introduction and Safety

1.1 What You Will Learn

This handbook guides you through the TURTLE Robotics Software Incubator (SI) curriculum. By the end of the nine-week series you will be able to write ROS 2 Python nodes, simulate robots in Gazebo, control a six-degree-of-freedom robot arm through software, and record training demonstrations for a neural network policy. Each chapter builds on the last, so work through them in order.

1.2 Equipment Overview

- SO-101 six-axis robot arm (follower unit)
- SO-101 leader arm for teleoperation (shared, one per station)
- 12V DC power supply
- USB-A to USB-C cable
- Laptop running Windows, macOS, or Linux

1.3 Safety Rules

- Keep fingers and loose clothing away from the arm during operation.
- Never exceed the joint angle limits listed in Chapter 7. The hardware will stall and may overheat.
- Power off the arm before connecting or disconnecting USB cables.
- If the arm makes a grinding noise or feels very hot, stop immediately and tell your instructor.
- Never leave the arm running and unattended.
- The arm has enough force to pinch. Do not try to stop a moving joint with your hand.

Chapter 2

Hardware Overview

2.1 The SO-101 Arm

The SO-101 is a low-cost, six-degree-of-freedom desktop robot arm developed by Hugging Face and the open-source robotics community. It uses Feetech STS3215 serial bus servos, which communicate over a half-duplex UART bus. The arm is designed for tabletop manipulation research and is capable of picking and placing small objects when properly calibrated and controlled.

The SO-101 comes as a follower arm (the arm that does the work) and a leader arm (a smaller arm you hold in your hand to command the follower via teleoperation). Both connect to the same computer over USB.

2.2 Joint Anatomy

The arm has six joints, numbered from base to tip. Each joint is driven by one servo and has a defined axis of rotation and range of motion.

Joint	Name	Axis	Range	What it does
J1	shoulder_pan	Z (yaw)	-180 to +180 deg	Rotates the whole arm left and right
J2	shoulder_lift	Y (pitch)	-90 to +90 deg	Tilts the upper arm forward and back
J3	elbow_flex	Y (pitch)	-155 to +155 deg	Bends the elbow
J4	wrist_flex	Y (pitch)	-90 to +90 deg	Tilts the wrist up and down
J5	wrist_roll	Z (yaw)	-180 to +180 deg	Spins the wrist
J6	gripper	X (roll)	0 to 90 deg	Opens and closes the gripper

Table 2.1 — SO-101 joint reference.

2.3 Wiring and Power

- Connect the 12V power supply to the arm base before anything else.
- Connect the USB cable from the arm base to your laptop.
- The power LED on the base should turn on. The servos will not move until software sends a command.
- For teleoperation, connect the leader arm to a second USB port on the same laptop.

Chapter 3

Software Setup

3.1 Installing the Environment

All software runs inside a conda environment called lerobot. On Windows you must use Miniforge Prompt, not PowerShell or Command Prompt, because Miniforge Prompt sets up the correct paths for conda.

Windows — Miniforge Prompt (Start > Miniforge3)

```
conda create -n lerobot python=3.10 -y
conda activate lerobot
conda install pytorch torchvision torchaudio cpuonly -c pytorch -y
cd C:\
git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[feetech]"
python -c "import lerobot; print(lerobot.__version__)"
```

macOS

```
brew install miniforge
conda init zsh   # then restart terminal
conda create -n lerobot python=3.10 -y
conda activate lerobot
cd ~ && git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[feetech]"
```

Linux

```
sudo apt update && sudo apt install -y git python3-pip ffmpeg
sudo usermod -aG dialout $USER   # log out and back in after this
conda create -n lerobot python=3.10 -y
conda activate lerobot
cd ~ && git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[feetech]"
```

3.2 Finding Your USB Port

Every command that talks to the arm requires the port name. Find it once and write it down.

Windows / macOS / Linux — Miniforge Prompt or terminal

```
conda activate lerobot
lerobot-find-port

# Unplug the arm, press Enter when prompted, then plug it back in.
# The script prints the port name — write it down.
# Run once for the follower arm, then again for the leader arm.
```

3.3 Calibrating the Arm

Calibration teaches the software the physical range of motion of each joint. You only need to do this once per arm. The calibration script will prompt you to move each joint slowly to its maximum position, then to its minimum position, then press Enter. Take your time and move all the way to each limit.

Windows

```
cd C:\lerobot && conda activate lerobot
# Follower arm:
lerobot-calibrate --robot.type=so101_follower --robot.port=COM4 --robot.id=so_follower
# Leader arm:
lerobot-calibrate --robot.type=so101_leader --robot.port=COM5 --robot.id=so_leader
```

macOS / Linux

```
cd ~/lerobot && conda activate lerobot
# Follower arm:
lerobot-calibrate --robot.type=so101_follower \
  --robot.port=/dev/ttyACM0 --robot.id=so_follower
# Leader arm:
lerobot-calibrate --robot.type=so101_leader \
  --robot.port=/dev/ttyACM1 --robot.id=so_leader
```

> *If a joint moves the wrong direction during teleoperation, ask your instructor about adjusting the drive_mode value in the calibration file.*

Chapter 4

ROS 2 Fundamentals

4.1 What is ROS 2?

ROS 2 (Robot Operating System 2) is a set of software libraries and tools for building robot applications. Despite the name, it is not an operating system — it runs on top of Linux, Windows, or macOS. ROS 2 gives you a standard way to write modular robot software where different pieces (sensor reading, motor control, path planning) run as separate programs that communicate with each other.

The version used in this course is ROS 2 Jazzy Jalisco, the current long-term support release. All simulation work runs inside a Docker container so you do not need to install ROS 2 directly on your laptop.

4.2 Nodes

A node is a single executable program that does one job. A well-designed ROS 2 application is made of many small nodes rather than one large program. Examples:

- A camera driver node that reads images from a USB camera and publishes them.
- A navigation node that reads sensor data and publishes velocity commands.
- A control node that reads velocity commands and sends them to motor controllers.

Nodes are written in Python or C++. In this course you will write Python nodes using the rclpy library.

Minimal ROS 2 Python node

```
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')   # name that appears in ros2 node list
        self.get_logger().info('Node started')

def main():
    rclpy.init()
    node = MyNode()
    rclpy.spin(node)   # keep running until Ctrl+C
    rclpy.shutdown()
```

4.3 Topics and Messages

Nodes communicate by publishing and subscribing to topics. A topic is a named channel with a specific message type. Any node can publish to a topic and any node can subscribe to it. Publishers and subscribers do not need to know about each other.

A message is a structured data type. ROS 2 provides many standard message types. The ones you will use most in this course:

Package	Type	Use
geometry_msgs	Twist	Linear and angular velocity commands
std_msgs	Float64	Single floating-point number (joint angles)
sensor_msgs	JointState	Position, velocity, effort for all joints
nav_msgs	Odometry	Position and velocity estimate from encoders

Table 4.1 — Common message types used in this course.

4.4 Publishers and Subscribers

A publisher sends messages on a topic at a rate you choose. A subscriber registers a callback function that runs every time a new message arrives. The same node can have multiple publishers and subscribers.

Publisher example

```
from geometry_msgs.msg import Twist

class DriveNode(Node):
    def __init__(self):
        super().__init__('drive_node')
        # Publisher: send Twist messages on /cmd_vel at queue depth 10
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        # Timer: call timer_callback every 0.1 seconds (10 Hz)
        self.timer = self.create_timer(0.1, self.timer_callback)

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 0.3   # move forward at 0.3 m/s
        self.pub.publish(msg)
```

Subscriber example

```
from nav_msgs.msg import Odometry

class OdomNode(Node):
    def __init__(self):
        super().__init__('odom_node')
        # Subscriber: call odom_callback when a message arrives
        self.sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

    def odom_callback(self, msg):
        x = msg.pose.pose.position.x
        self.get_logger().info(f'x = {x:.3f} m')
```

4.5 The ROS 2 Graph

The collection of all running nodes and the topics connecting them is called the ROS 2 graph. You can inspect the graph at any time while nodes are running. rqt_graph opens a visual diagram of the graph in a window. Understanding the graph is the fastest way to debug a system where something is not receiving data.

4.6 Useful CLI Commands

Command	What it does
`ros2 node list`	List all running nodes
`ros2 node info <node>`	Show publishers, subscribers, services for a node
`ros2 topic list`	List all active topics
`ros2 topic echo <topic>`	Print messages arriving on a topic
`ros2 topic info <topic>`	Show type, publisher count, subscriber count
`ros2 topic pub --once <topic> <type> <data>`	Publish one message from the terminal
`ros2 topic hz <topic>`	Show how fast messages are arriving
`rqt_graph`	Open a visual graph of nodes and topics

Table 4.2 — Essential ROS 2 command-line tools.

Chapter 5

Gazebo Simulation

5.1 What is Gazebo?

Gazebo is a robot simulation environment. It models physics (gravity, friction, inertia), sensors (cameras, IMUs, encoders), and actuators (motors, joints). You can test your ROS 2 code in Gazebo before running it on real hardware, which saves time and prevents crashes.

This course uses Gazebo Harmonic, the current long-term support release. Gazebo communicates with ROS 2 through the ros_gz bridge, which automatically forwards Gazebo topics to ROS 2 topics and vice versa.

5.2 The TURTLE SI Workspace

The SI workspace contains two ROS 2 packages inside ros2_ws/src/:

- **turtle_gazebo** — World files, robot SDF models, and launch files.
- **turtle_control** — Python control nodes. This is where you write your code.

SDF (Simulation Description Format) is the file format Gazebo uses to describe robots and worlds. The turtle_bot and SO-101 arm are both defined as SDF files in the models/ directory.

5.3 Launching the Simulation

All commands below are run inside the Docker container. Open a terminal inside the container with:

```
docker compose exec ros bash
```

Then build the workspace once:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

Launch the simulation:

```
# Robot only:
ros2 launch turtle_gazebo simulation.launch.py

# Robot and SO-101 arm:
ros2 launch turtle_gazebo simulation_with_arm.launch.py
```

Gazebo will open and load an empty world. The turtle_bot spawns after about three seconds. The SO-101 arm spawns at x=1.0 m (one meter to the right of the robot) when using the arm launch file.

5.4 Interacting with the World

While Gazebo is running you can check what topics exist:

```
# Open a second terminal in the container:
docker compose exec ros bash
source ~/ros2_ws/install/setup.bash

ros2 topic list
```

You should see topics for the turtle_bot and, if you launched the arm, for each arm joint. The next two chapters explain what to do with those topics.

Driving the Turtle Bot

6.1 Differential Drive

The turtle_bot uses differential drive, the same locomotion system found on most wheeled robots including the Roomba and the TurtleBot 3. Two independently driven wheels are mounted on opposite sides of the robot. Two passive caster balls provide stability at the front and rear.

By varying the speed of the two wheels you can produce any combination of forward motion and rotation. If both wheels spin at the same speed the robot goes straight. If the left wheel spins faster than the right the robot turns right, and vice versa. If the wheels spin in opposite directions the robot rotates in place.

6.2 The cmd_vel Topic

The Gazebo differential drive plugin listens on the topic /model/turtle_bot/cmd_vel for velocity commands. Any node that publishes to this topic can drive the robot.

6.3 Twist Messages

The geometry_msgs/Twist message has two fields, each a three-component vector:

- **linear.x** — Forward velocity in meters per second. Positive = forward, negative = backward.
- **linear.y** — Sideways velocity. Differential drive robots cannot move sideways, so leave this at zero.
- **linear.z** — Vertical velocity. Not applicable for ground robots. Leave at zero.
- **angular.x** — Roll rate. Not applicable. Leave at zero.
- **angular.y** — Pitch rate. Not applicable. Leave at zero.
- **angular.z** — Rotation rate in radians per second. Positive = turn left, negative = turn right.

6.4 Sending Commands Manually

```
# Drive forward at 0.2 m/s:
ros2 topic pub --once /model/turtle_bot/cmd_vel geometry_msgs/Twist \
  "{linear: {x: 0.2}, angular: {z: 0.0}}"

# Turn left at 0.5 rad/s:
ros2 topic pub --once /model/turtle_bot/cmd_vel geometry_msgs/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.5}}"

# Stop:
ros2 topic pub --once /model/turtle_bot/cmd_vel geometry_msgs/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

The --once flag sends a single message and exits. Without it the command keeps publishing at 1 Hz until you press Ctrl+C.

6.5 Reading Odometry

The robot publishes its estimated position and velocity on /model/turtle_bot/odom. This data comes from integrating wheel encoder counts and is accurate for short distances on flat ground.

```
ros2 topic echo /model/turtle_bot/odom

# You will see output like:
# pose:
#   pose:
#     position:
#       x: 0.312
#       y: -0.001
#       z: 0.1
#     orientation:
#       x: 0.0
#       y: 0.0
#       z: 0.245
#       w: 0.969
```

Orientation is given as a quaternion (x, y, z, w). Chapter 9 explains how to convert this to a yaw angle in radians.

Arm Control via ROS 2 Topics

7.1 Joint Position Control

Each joint of the SO-101 arm is controlled by a Gazebo JointPositionController plugin. The controller uses a PID loop to move the joint smoothly to a target angle and hold it there. You command a joint by publishing its target angle in radians to a topic. The controller handles the rest.

This is open-loop from the user's perspective: you say 'go to 0.5 radians' and the controller moves there. You do not need to send continuous commands.

7.2 Radians vs Degrees

All joint commands use radians, not degrees. The conversion is straightforward:

```
import math

# Convert degrees to radians:
angle_rad = math.radians(45)   # = 0.785
angle_rad = math.radians(90)   # = 1.571
angle_rad = math.radians(180)  # = 3.142

# Common values to memorize:
# 0 rad   =   0 deg  (neutral / home)
# 0.785   =  45 deg
# 1.571   =  90 deg
# 3.142   = 180 deg  (half rotation)
```

7.3 Joint Topics Reference

Joint	Topic	Range (rad)	Notes
shoulder_pan	`/so101/shoulder_pan`	-3.14 to +3.14	+1.57 = 90 deg left
shoulder_lift	`/so101/shoulder_lift`	-1.57 to +1.57	0 = straight up
elbow_flex	`/so101/elbow_flex`	-2.70 to +2.70	+1.0 = bent inward
wrist_flex	`/so101/wrist_flex`	-1.57 to +1.57	+1.0 = tilted forward
wrist_roll	`/so101/wrist_roll`	-3.14 to +3.14	+3.14 = full spin
gripper	`/so101/gripper`	0 to +1.57	0 = open, 1.57 = closed

Table 7.1 — Arm joint topics and ranges.

7.4 Sending Joint Commands

From the terminal:

```
# Pan the base 45 degrees left:
ros2 topic pub --once /so101/shoulder_pan std_msgs/Float64 'data: 0.785'

# Lift the shoulder forward:
ros2 topic pub --once /so101/shoulder_lift std_msgs/Float64 'data: -0.5'

# Close the gripper:
ros2 topic pub --once /so101/gripper std_msgs/Float64 'data: 1.57'

# Open the gripper:
ros2 topic pub --once /so101/gripper std_msgs/Float64 'data: 0.0'
```

From Python (inside arm_control.py):

```
from std_msgs.msg import Float64

class ArmControl(Node):
    def __init__(self):
        super().__init__('arm_control')
        # One publisher per joint:
        self.pubs = {
            'shoulder_pan':  self.create_publisher(Float64, '/so101/shoulder_pan',  10),
            'shoulder_lift': self.create_publisher(Float64, '/so101/shoulder_lift', 10),
            'elbow_flex':    self.create_publisher(Float64, '/so101/elbow_flex',    10),
            'wrist_flex':    self.create_publisher(Float64, '/so101/wrist_flex',    10),
            'wrist_roll':    self.create_publisher(Float64, '/so101/wrist_roll',    10),
            'gripper':       self.create_publisher(Float64, '/so101/gripper',       10),
        }

    def send(self, joint, angle_rad):
        msg = Float64()
        msg.data = float(angle_rad)
        self.pubs[joint].publish(msg)
```

7.5 Reading Joint States

```
ros2 topic echo /so101/joint_states

# Output shows name, position (rad), velocity, and effort for each joint.
```

Week 6 Lab — Open-Loop Drive

8.1 Objective

Write a ROS 2 Python node that drives the turtle_bot through a predefined sequence of movements using open-loop control. By the end you will understand how to create a publisher, use a timer callback, and translate motion goals into Twist messages.

8.2 Background

Open-loop control means you send commands based on time alone, without checking whether the robot actually reached its goal. It is simple and easy to implement, but errors accumulate over time because you ignore feedback. For short, predictable tasks in a clean environment it works well enough to be useful.

The node you write will use a timer that fires at 10 Hz. Each time the timer fires it checks how much time has elapsed and publishes the appropriate Twist command for that phase of the motion sequence.

8.3 Instructions

Open the file ros2_ws/src/turtle_control/turtle_control/drive_turtle.py in a text editor. The file already has the class structure and imports. Your job is to complete the timer_callback method.

The sequence your node should produce:

- Drive forward at 0.3 m/s for 3 seconds.
- Turn left at 0.5 rad/s for 2 seconds.
- Stop and hold position.

drive_turtle.py — timer_callback skeleton

```
# The structure of timer_callback — fill in the Twist values:
def timer_callback(self):
    msg = Twist()
    self.elapsed += 0.1   # timer fires every 0.1 s

    if self.elapsed < 3.0:
        msg.linear.x  = ___   # forward
        msg.angular.z = ___
    elif self.elapsed < 5.0:
        msg.linear.x  = ___
        msg.angular.z = ___   # turn left
    else:
        msg.linear.x  = 0.0   # stop
        msg.angular.z = 0.0

    self.cmd_pub.publish(msg)
```

After editing, rebuild and run:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run turtle_control drive_turtle
```

8.4 Going Further

- Add a fourth phase that drives backward to approximately the starting position.
- Make the robot drive a square: four forward segments separated by 90-degree turns.
- Vary the speed during the forward phase and observe how the robot's final position changes.
- Use ros2 topic echo /model/turtle_bot/odom to watch the position change while the node runs.

Week 9 Lab — Closed-Loop Control

9.1 Objective

Extend drive_turtle.py to subscribe to the odometry topic and use position feedback to stop at precise targets. You will drive exactly one meter forward and then turn exactly ninety degrees, stopping when the measured pose reaches the goal rather than relying on a timer.

9.2 Background: Odometry

Odometry is the process of estimating a robot's position by counting how far each wheel has turned. The robot's firmware integrates wheel encoder pulses into an x, y, and heading estimate and publishes it as a nav_msgs/Odometry message.

The orientation is given as a quaternion (x, y, z, w) because quaternions avoid the gimbal lock problem that affects Euler angles. For a ground robot that only rotates about the vertical axis, you only need the yaw angle, which you extract using the atan2 formula below.

Quaternion to yaw conversion

```
import math

def quaternion_to_yaw(q):
    """Extract yaw (rotation about Z axis) from a ROS quaternion."""
    return math.atan2(
        2.0 * (q.w * q.z + q.x * q.y),
        1.0 - 2.0 * (q.y ** 2 + q.z ** 2)
    )
    # Returns a value in radians: -pi to +pi
    # 0 = facing forward, pi/2 = facing left, -pi/2 = facing right
```

9.3 Instructions

Add an odometry subscriber to your drive_turtle.py node and rewrite the control logic to use position and heading feedback instead of elapsed time. Record the starting pose in __init__ when the first odometry message arrives, then compare the current pose to that starting point in each timer callback.

Odometry subscriber skeleton

```
from nav_msgs.msg import Odometry
import math

# Add to __init__:
self.odom_sub = self.create_subscription(
    Odometry, '/model/turtle_bot/odom', self.odom_callback, 10)
self.current_x   = 0.0
self.current_y   = 0.0
self.current_yaw = 0.0
self.start_x     = None   # set on first odom message
self.start_yaw   = None

# Add odom_callback method:
def odom_callback(self, msg):
    self.current_x = msg.pose.pose.position.x
    self.current_y = msg.pose.pose.position.y
    q = msg.pose.pose.orientation
    self.current_yaw = math.atan2(
        2*(q.w*q.z + q.x*q.y),
        1 - 2*(q.y**2 + q.z**2)
    )
    if self.start_x is None:
        self.start_x   = self.current_x
        self.start_yaw = self.current_yaw
```

Closed-loop timer_callback

```
# Rewrite timer_callback to use feedback:
def timer_callback(self):
    if self.start_x is None:
        return   # wait for first odom message

    msg = Twist()
    dist = abs(self.current_x - self.start_x)   # simplification: assumes straight line
    yaw_change = self.current_yaw - self.start_yaw

    if dist < 1.0:                   # phase 1: drive 1 meter
        msg.linear.x = 0.2
    elif abs(yaw_change) < math.pi/2: # phase 2: turn 90 degrees
        msg.angular.z = 0.3
    # else: stop (msg stays all zeros)

    self.cmd_pub.publish(msg)
```

9.4 Going Further

- Handle the case where the robot starts at a non-zero x position by computing distance from start_x rather than from zero.
- Drive a precise square: use feedback to complete each side and each turn before moving to the next.
- Add a proportional controller: slow down as the robot approaches the goal distance instead of stopping abruptly.

- Compare the final odometry position to the expected position. How much error accumulates over one meter?

Real Hardware — Teleoperation

10.1 Teleop Setup

Teleoperation lets you move the follower arm by physically moving the leader arm. The software reads the leader joint positions at 30 Hz and sends them to the follower in real time. Before running teleop, confirm that both arms are calibrated.

Windows

```
cd C:\lerobot && conda activate lerobot
python -m lerobot.scripts.control_robot teleoperate \
  --robot-path lerobot/configs/robot/so101_follower.yaml \
  --robot-overrides "~cameras" \
  robot.port=COM4 leader_arms.main.port=COM5
```

macOS / Linux

```
cd ~/lerobot && conda activate lerobot
python -m lerobot.scripts.control_robot teleoperate \
  --robot-path lerobot/configs/robot/so101_follower.yaml \
  --robot-overrides "~cameras" \
  robot.port=/dev/ttyACM0 leader_arms.main.port=/dev/ttyACM1
```

All six joints should move smoothly and mirror the leader arm without delay. Press Ctrl+C to stop. If the arm makes clicking sounds or a joint does not follow, re-run calibration for that arm.

10.2 Recording Demonstrations

To train a neural network policy you need a dataset of demonstrations. Each demonstration (episode) is a recording of you teleooperating the arm to complete a task. Aim for at least 50 episodes of consistent, smooth motion. Start from the same position every episode and complete the full task each time.

Log in to Hugging Face before recording:

```
huggingface-cli login
```

macOS / Linux recording command

```
# Replace YOUR_USERNAME with your Hugging Face username:
python -m lerobot.scripts.control_robot record \
  --robot-path lerobot/configs/robot/so101_follower.yaml \
  --robot-overrides "~cameras" \
  robot.port=/dev/ttyACM0 leader_arms.main.port=/dev/ttyACM1 \
  --fps 30 \
  --repo-id YOUR_USERNAME/my_dataset \
  --tags so101 \
  --warmup-time-s 2 \
  --episode-time-s 30 \
  --reset-time-s 5 \
  --num-episodes 50
```

Training a Policy

11.1 What is Imitation Learning?

Imitation learning is a branch of machine learning where a robot learns to perform a task by observing demonstrations rather than through trial and error. Your recorded episodes form the training dataset. A neural network learns to map observed joint states (and optionally camera images) to joint commands by minimizing the difference between its predictions and your recorded actions.

This course uses the ACT (Action Chunking with Transformers) policy, a state-of-the-art imitation learning method developed at Stanford. ACT predicts a chunk of future actions at once rather than one action at a time, which makes it more robust to noise.

11.2 Running Training

Training is computationally intensive. With a dedicated GPU it takes one to two hours. On a CPU it takes eight to twelve hours. Your instructor may run training overnight on a lab machine.

```
# Run from inside the lerobot folder with the environment active:
python -m lerobot.scripts.train \
  --policy-path lerobot/configs/policy/act.yaml \
  --dataset-repo-id YOUR_USERNAME/my_dataset \
  --output-dir outputs/train/act_so101 \
  --training.num-steps 80000 \
  --wandb.enable false
```

11.3 Evaluating the Policy

Once training finishes, run the trained policy on the real arm to see if it successfully completes the task autonomously.

```
python -m lerobot.scripts.control_robot eval \
  --robot-path lerobot/configs/robot/so101_follower.yaml \
  --robot-overrides "~cameras" robot.port=/dev/ttyACM0 \
  --policy-path outputs/train/act_so101/checkpoints/last/pretrained_model \
  --num-eval-episodes 5
```

If the policy does not work reliably, the most common fixes are collecting more demonstrations, making the demonstrations more consistent, or reducing the complexity of the task.

Sim Training Pipeline

This chapter explains how to collect demonstration data using the Gazebo simulation, train an ACT imitation-learning policy on that data, and then transfer the trained policy to the real SO-101 arm. The workflow is designed so that anyone — regardless of whether they have access to the physical hardware — can contribute demonstrations and improve the shared policy.

12.1 System Overview

The pipeline has three stages: (1) **Collect** — teleop the arm in Gazebo and record joint state sequences as episodes; (2) **Train** — feed those episodes to the ACT policy, which learns to mimic the demonstrated motions; (3) **Eval** — deploy the trained policy to the real hardware and test it. All three stages share the same joint naming convention and topic structure, so data and weights move between sim and hardware without modification.

Modes of Operation

Mode	Hardware Needed	Teleop Method	Use Case
At-home (keyboard)	PC only	keyboard_teleop	Collect data without lab access
In-lab (leader arm)	PC + leader arm	leader_bridge	High-quality, natural demonstrations

Table 12.1 — Sim training pipeline modes.

12.2 At-Home Mode: Keyboard Teleop

When you do not have access to the leader arm, use the keyboard teleop node. It lets you drive all six joints of the simulated arm directly from the terminal. Demonstrations collected this way are less natural than leader-arm demos, but are still valid training data for learning simple pick-and-place motions.

Step 1 — Launch the Simulation

```
# Inside the Docker container:
docker compose exec ros bash
source install/setup.bash

# Start Gazebo + keyboard teleop + recorder in one command:
ros2 launch turtle_gazebo sim_teleop.launch.py

# This opens three windows automatically:
#   1. Gazebo (the 3-D simulation)
#   2. keyboard_teleop (joint controls)
#   3. sim_recorder (episode recorder)
```

Step 2 — Drive the Arm

Click on the keyboard teleop window. The six joints are mapped to keys 1–6. Use the up and down arrow keys to change the selected joint's angle by 0.05 rad per keypress. Press H at any time to return all joints to their home position.

```
■■■
```

```
# Replace /dev/ttyACM1 with your actual leader arm port:
ros2 launch turtle_gazebo sim_teleop.launch.py \
    mode:=leader port:=/dev/ttyACM1

# Windows example:
ros2 launch turtle_gazebo sim_teleop.launch.py mode:=leader port:=COM5
```

The launch file starts Gazebo, connects the leader bridge, and opens the sim_recorder window. The bridge polls the leader arm at 30 Hz and publishes each joint position to the corresponding Gazebo topic.

Step 2 — Collect Demonstrations

Hold the leader arm and move it through the task you want to teach. The simulated arm follows in real time. Press SPACE in the recorder window to start and stop each episode. Aim for smooth, deliberate movements. Collect at least 20 episodes; 50 or more is better.

> *You can switch between keyboard and leader mode for the same dataset. Both produce identical JSON files and are interchangeable during training.*

12.4 Training the ACT Policy

Once you have collected enough episodes, train the ACT policy on the dataset. Training happens on a GPU-enabled machine (the SI lab PC or a cloud service). You do not need the simulation running during training.

Step 1 — Convert the Dataset

```
conda activate lerobot

# Convert the raw JSON episodes to lerobot's dataset format:
python ~/ros2_ws/src/turtle_control/scripts/convert_to_lerobot.py \
    --input ~/sim_dataset \
    --output ~/lerobot_dataset

# This creates ~/lerobot_dataset/ in the HuggingFace datasets format
# that lerobot's training script expects.
```

Step 2 — Run Training

```
cd ~/lerobot

python -m lerobot.scripts.train \
    --policy-path lerobot/configs/policy/act.yaml \
    --dataset-repo-id local/sim_dataset \
    --output-dir outputs/train/act_so101_sim \
    --training.num_epochs 100

# Training logs loss every few steps.
# A well-converged policy shows loss < 0.05 by epoch 80-100.
# Checkpoints are saved in outputs/train/act_so101_sim/checkpoints/
```

Training 50 episodes for 100 epochs takes approximately 20–30 minutes on an NVIDIA RTX 3060 or equivalent. If you do not have a local GPU, contact your instructor about using the shared lab workstation.

12.5 Deploying to the Real Arm (Sim2Real)

After training, deploy the policy to the real SO-101 follower arm to test whether the sim-trained behavior transfers. This is called sim-to-real transfer (sim2real). Joint names and ranges are identical between the simulation and the hardware, so no additional adaptation is required.

Step 1 — Connect the Follower Arm

Connect the SO-101 follower arm to the PC. Confirm its port with lerobot-find-port. The follower arm must already be calibrated.

Step 2 — Run Evaluation

```
conda activate lerobot

python -m lerobot.scripts.control_robot eval \
    --robot-path lerobot/configs/robot/so101_follower.yaml \
    --robot-overrides robot.port=/dev/ttyACM0 \
    --policy-path outputs/train/act_so101_sim/checkpoints/last/pretrained_model \
    --num-eval-episodes 5

# The arm moves autonomously through the task for 5 trials.
# Observe whether it successfully completes the task each time.
```

A success rate of 60–80 % on the first real-world test is typical for a sim-trained policy. Common causes of failure are: inconsistent demonstrations, insufficient episodes, or tasks that depend on visual feedback the simulation does not yet model accurately.

12.6 Improving the Policy

If the initial success rate is low, try the following:

- **More demonstrations.** Add 20–30 episodes to the dataset and retrain. More data is the most reliable fix.
- **Consistent demonstrations.** Review your episodes for outliers — episodes where you made large mistakes or recovered awkwardly. Delete them and recollect.
- **Simpler task.** If the task requires precise visual alignment, simplify it so that joint positions alone are sufficient to complete it.
- **More epochs.** Try 200 epochs if the loss curve had not yet flattened at 100.

Troubleshooting and Glossary

12.1 Common Errors

Problem	Fix
colcon build fails: package not found	Run: rosdep update && rosdep install --from-paths src -i -y
Gazebo opens but robot does not appear	Wait 5 seconds. The spawn is delayed. Check the terminal for spawn error messages.
/model/turtle_bot/cmd_vel not in ros2 topic list	The robot has not spawned yet, or Gazebo crashed. Restart the launch file.
docker compose build is very slow	Normal on first build. The image is approximately 2 GB. Subsequent builds use the cache
error: src refspec main does not match any	Your local branch is named master, not main. Use: git push -u origin master --force
Arm joint moves the wrong direction	Adjust drive_mode in the calibration YAML file. Ask your instructor.
conda: command not found on Windows	You are not in Miniforge Prompt. Open Start > Miniforge3 > Miniforge Prompt.
USB port not found	Check that the 12V power is connected, try a different USB cable, try a different port.
Teleop: arm does not move	Confirm calibration ran successfully for both arms. Check that port numbers are correct.
Training loss does not decrease	Collect more demonstrations. Ensure demonstrations are consistent and complete.
keyboard_teleop: terminal shows garbled output	Make sure no other process is reading stdin. Run in its own dedicated terminal window.
leader_bridge: lerobot not installed error	Activate the conda environment before launching: conda activate lerobot
sim_recorder: no frames recorded	Check that /so101/joint_states is being published: ros2 topic echo /so101/joint_states
sim2real: arm overshoots target position	The PID gains on the real arm may differ from simulation. Recalibrate or tune in control_r
xterm not found when using launch file	Install xterm: sudo apt install xterm, or run keyboard_teleop and sim_recorder in separate

12.2 Glossary

- **ACT** — Action Chunking with Transformers. The imitation learning policy used in this course.
- **Callback** — A function that runs automatically when an event occurs, such as a new message arriving on a topic.
- **colcon** — The build tool used to compile ROS 2 packages.
- **Differential drive** — A locomotion system using two independently driven wheels to produce forward motion and rotation.
- **Docker** — A containerization platform that packages software and its dependencies into isolated units.
- **Gazebo** — A robot simulation environment that models physics, sensors, and actuators.
- **IMU** — Inertial Measurement Unit. A sensor that measures acceleration and rotation rate.
- **Joint** — A single motorized axis of rotation or translation on a robot arm.

- **Node** — A single ROS 2 executable program with one focused responsibility.
- **Odometry** — Position estimation by integrating wheel encoder data over time.
- **PID** — Proportional-Integral-Derivative. A feedback control algorithm that drives a measured value toward a target.
- **Publisher** — A ROS 2 object that sends messages on a topic.
- **Quaternion** — A four-component representation of 3D rotation that avoids gimbal lock.
- **rclpy** — The ROS 2 Python client library.
- **ROS 2** — Robot Operating System 2. A framework for writing modular robot software.
- **SDF** — Simulation Description Format. The XML format used to describe robots and worlds in Gazebo.
- **Subscriber** — A ROS 2 object that receives messages from a topic and calls a callback function.
- **Topic** — A named communication channel in ROS 2 with a specific message type.
- **Twist** — A ROS 2 message type representing linear and angular velocity.
- **Yaw** — Rotation about the vertical axis. For a ground robot this is the heading angle.